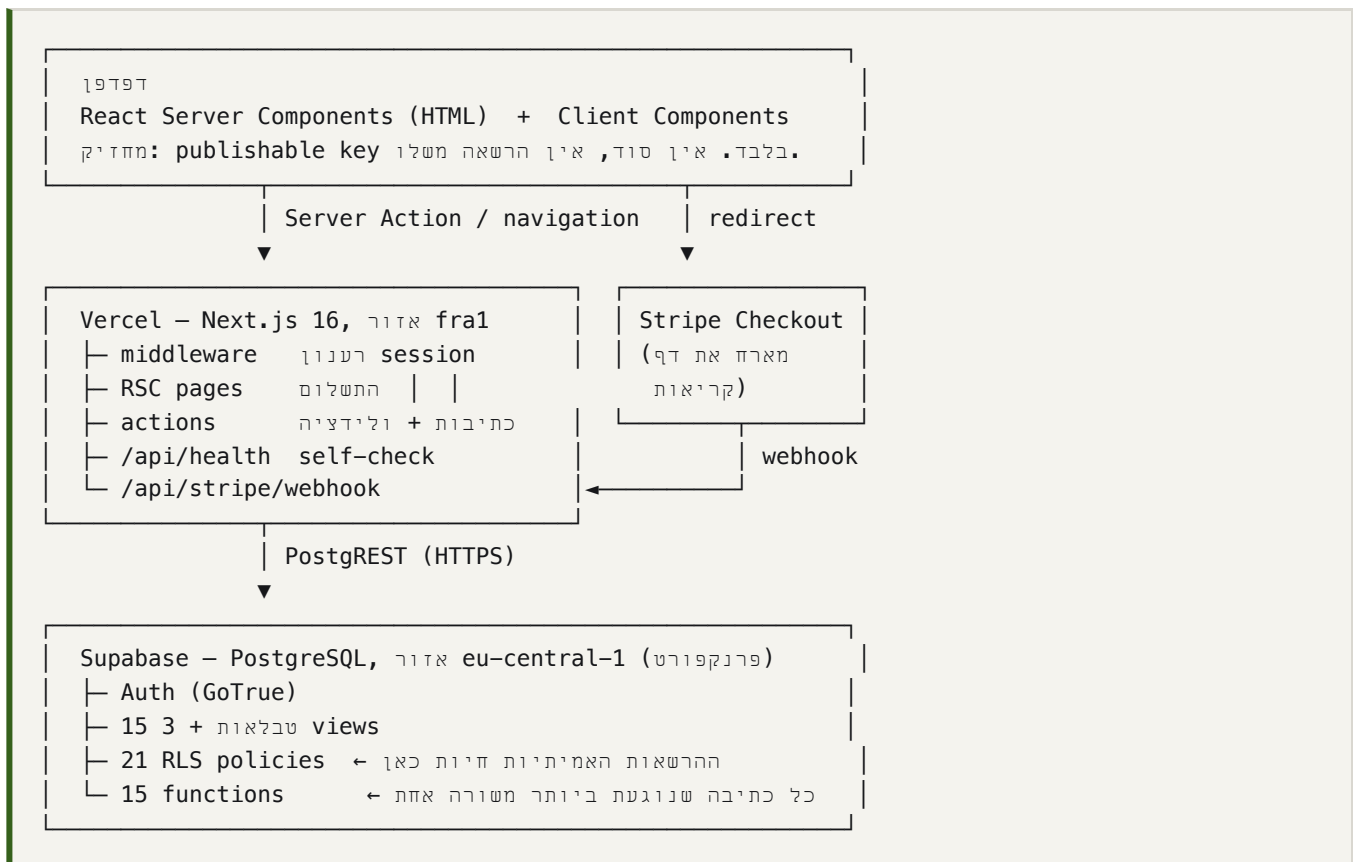


מסמך ארכיטקטורה ותכנון טכני – HavuGym Crew

מכסה את סעיפים 3 ו-4 בבריף: ארכיטקטורת המערכת והתכנון הטכני המפורט.

1. תמונת המערכת



1.1 החלטת אזור

Supabase בפרנקפורט; Vercel פותח פרויקטים חדשים ב- **iad1** (וושנינגטון). ללא התערבות, **כל שאילתה** הייתה חוצה את האוקיינוס – כ-100ms לכיוון, פעם לכל שאילתה, בבקשה שעושה כמה. **vercel.json** מצמיד ל- **fra1**. זו ההחלטה היחידה הגדולה ביותר של latency במערכת, והיא שורת קונפיגורציה שנכתבה לפני שורת קוד אחת.

1.2 למה Supabase ולא Postgres גולמי

לא בגלל שהוא "מסד נתונים מנוהל", אלא בגלל **Auth + RLS באותו מקום**. הצירוף הזה מאפשר להעביר את שכבת ההרשאות מהאפליקציה למסד. משמעות מעשית: שאילתה שנשכח בה **WHERE user_id = ...** מחזירה **אפס שורות**, לא את כולן. באפליקציה קלאסית זו פרצה; כאן זה באג ריק.

2. מבנה התיקיות

havugym-crew/	
├─ src/	
│ └─ app/	
│ └─ page.tsx	דף נחיתה
│ └─ login/	התחברות והרשמה
│ └─ onboarding/	יצירה או הצטרפות לחבורה
│ └─ (app)/	session כל מה שדורש – route group
│ └─ layout.tsx	מעטפת, ניווט, יתרת קריאטיב
│ └─ feed/	של החבורה + המלצה feed-ה
│ └─ log/	רישום אימון
│ └─ crew/	חברים, אתגר, תחרות
│ └─ shop/	חנות + רכישת חבילות
│ └─ me/	ledger, פרופיל, סטטיסטיקה
│ └─ workouts/[id]/	אימון בודד
│ └─ api/	
│ └─ health/	בדיקת בריאות פריסה
│ └─ stripe/webhook/	קליטת אירועי תשלום
├─ components/	משתף UI
├─ lib/	
│ └─ domain/	אין שטון DB, לוגיקה עסקית טהורה. אין ←
│ └─ actions/	← Server Actions (כתיבות)
│ └─ services/	← (service role) פעולות מיוחדות
│ └─ supabase/	← clients: server / browser / admin שלושה
│ └─ validation/	← סכמות Zod
│ └─ queries.ts	← כל הקריאות
│ └─ database.types.ts	← (npx supabase gen types) נוצר מהסכמה
└─ middleware.ts	
├─ supabase/migrations/	מיגרציות, מסודרות 11
├─ tests/	unit + integration
├─ e2e/	Playwright
└─ docs/	המסמכים המוגשים

כלל הארגון: lib/domain לא מייבא כלום מ- lib/supabase. הכיוון תמיד חד-סטרי – שכבות חיצוניות מייבאות פנימה, לעולם לא להפך. זה מה שמאפשר לבדוק את כל הלוגיקה העסקית בלי מסד נתונים.

2.1 העמודים באפליקציה

13 נתיבים. אחד עשר דפים ושני route handlers.

נתיב	דורש session	סוג	תפקיד
/	לא	RSC	דף נחיתה. מפנה ל- /feed אם יש session
/login	לא	RSC + Client	התחברות והרשמה, טאב אחד לכל אחד
/onboarding	כן	RSC + Client	יצירת חבורה או הצטרפות בקוד. חסום למי שכבר בחבורה
/feed	כן	RSC + Client	באנר "מי במכון עכשיו" + ה-feed + המלצה + pagination
/log	כן	RSC + Client	רישום אימון עם ציון חזוי חי
/workouts/[id]	כן	RSC	אימון בודד, סטים מלאים, מחיקה לבעלים
/crew	כן	RSC	חברים, קוד הזמנה, אתגר, תחרות. מפעיל סגירת שבועות
/shop	כן	RSC + Client	קטלוג קוסמטי, רכישה, ציוד, חבילות קריאטין
/shop/success	כן	RSC	חזרה מ-Stripe; מזכה אם ה-webhook טרם הגיע
/me	כן	RSC + Client	פרופיל, סטטיסטיקה, ledger, מחליף חבורות, עזיבה, התנתקות
/api/health	לא	Route	בדיקת פריסה – קונפיגורציה, מסד, ואכיפת RLS
/api/stripe/webhook	לא	Route	קליטת אירועי תשלום עם אימות חתימה

רוב הדפים הם Server Components. לקוח נטען רק היכן שיש אינטראקטיביות אמיתית – הלוגר, טפסי ההתחברות וההצטרפות, כפתורי החנות, מחליף החבורות והפעולות ההרסניות.

2.2 מבנה הקומפוננטות

עיקרון: ה-RSC שואב את הנתונים ומעביר אותם למטה. Client Component מקבל props ומחזיק state – הוא לעולם לא שולף בעצמו.

קומפוננטה	סוג	תפקיד
app-nav.tsx	Client	ניווט. Client רק בגלל <code>usePathname</code> להדגשת הטאב
workout-card.tsx	Server	כרטיס אימון ב-feed. ללא state, ולכן ללא JS בצד הלקוח
icons.tsx	Server	14 אייקוני SVG על רשת 24, יורשים <code>currentColor</code>
form.tsx	Client	SubmitButton (דרך <code>useFormStatus</code>), <code>FieldError</code> , <code>FormError</code>
invite-code.tsx	Client	הצגה והעתקה. נכשל בשקט כשאין <code>clipboard</code>
crew-switcher.tsx	Client	החלפת חבורה פעילה + <code>router.refresh()</code>
destructive-button.tsx	Client	אישור דו-שלבי במקום <code>window.confirm</code>
arrival-banner.tsx	Client	"מי במכון עכשיו". האות מחושב בשרת ויורד מוכן – ראו למטה
workout-logger.tsx	Client	ה-state המורכב היחיד באפליקציה
login-form.tsx	Client	שני <code>useActionState</code> נפרדים, כדי ששגיאות לא יעברו בין הטאבים
onboarding-form.tsx	Client	אותו דפוס, ליצירה מול הצטרפות
shop-actions.tsx	Client	ItemButton – קנייה או ציוד לפי מצב הבעלות
buy-pack.tsx	Client	פתיחת Stripe Checkout והפניה

למה האות של ההגעות מחושב בשרת: חישוב מחדש בלקוח מחייב `new Date()` בזמן רינדור, מה שמייצר מחרוזת אחרת בשרת ובדפדפן – hydration mismatch בקומפוננטה שכל תפקידה הוא להיות אמינה. השרת מחשב את הכותרת, תת-הכותרת והדקות שנותרו, והלקוח אחראי רק לכפתורים ולרענון.

למה polling ולא Realtime: websocket הוא עוד חיבור, עוד מצב כשל ועוד נושא במסמך האבטחה – בתמורה להשהיה שאיש מהמחליטים אם ללכת לחדר כושר אינו מסוגל להבחין בה. דקה אינה איחור.

למה SubmitButton היא קומפוננטה נפרדת: `useFormStatus` קורא את מצב ה-`<form>` העוטף. קריאה שלו מתוך הקומפוננטה שמרנדרת את הטופס מחזירה `pending: false` תמיד – הוא חייב להיות בתוך הטופס, לא סביבו.

3. מסד הנתונים

3.1 הישויות

#	טבלה	תפקיד
1	profiles	פרופיל לכל משתמש. נוצר בטריגר על auth.users
2	havras	החבורה – גבול ה-tenant
3	havura_members	חברות + תפקיד. הציר שכל policy נשען עליו
4	exercises	קטלוג 660 תרגילים, קריאה בלבד
5	workouts	אימון שנרשם
6	workout_sets	הסטים של אימון
7	challenges	האתגר השיתופי השבועי
8	challenge_progress	התקדמות + paid_at
9	competitions	התחרות השבועית
10	competition_results	דירוג וזכייה, נכתב בסגירה
11	createine_ledger	append-only. מקור האמת של המטבע
12	shop_items	קטלוג קוסמטי
13	inventory	בעלות + מה מצויד
14	orders	רכישות בכסף אמיתי
15	arrivals	"אני במכון" – המשטח הפרוספקטיבי היחיד

3.2 יחסים

```
auth.users —1:1→ profiles —< havura_members >— havuras
                                |< workouts < workout_sets >— exercises
                                |< createine_ledger
                                |< inventory >— shop_items
                                |< orders
havras —< challenges < challenge_progress
havras —< competitions < competition_results
```

3.3 החלטות תכנון שראוי להסביר

א. המטבע הוא ledger, לא מספר. profiles.createine_balance הוא createine_ledger; cache הוא האמת. כל שורה שומרת balance_after, כך שהיתרה ניתנת לביקורת מתוך ה-ledger לבדו. פונקציה אחת בלבד – apply_createine() – כותבת לשניהם, בטריג'ר אחת, תחת UPDATE ... FOR SELECT. למה: מספר בודד לא עונה על "למה יש לי 825?". Ledger עונה, וגם הופך את האינפורמנט לבדיקה בשורה אחת: cache == SUM(ledger).

ב. Idempotency היא אינדקס, לא קוד.

```
create unique index creatine_ledger_ref_uniq
on creatine_ledger (user_id, reason, ref_type, ref_id)
where ref_id is not null;
```

webhook שנשלח פעמיים, כפתור שנלחץ פעמיים, וסגירת שבוע שרצה שוב – כולם מתנגשים באינדקס הזה ומפסידים אותו רעיון על workouts(user_id, source, external_id) הופך את מתאם Heavy לבטוח **לפני** שנכתב.

ג. Enums במקום CHECK על טקסט. מתעדים את עצמם, זולים לאינדוקס, וטעות כתיב הופכת לשגיאת מיגרציה במקום לשורה שקטה ושגויה.

ד. FK מורכב שאוכף כלל עסקי. inventory מחזיקה item_kind בשכפול מ-shop_items, עם FK על item_id, (item_kind, זה מאפשר:

```
create unique index on inventory (user_id, item_kind) where equipped;
```

"פריט מצויד אחד מכל סוג" נאכף על ידי המסד – לא על ידי הקוד שבמקרה קרא ל- equip().

ה. competitions לא יכולה להיות במצב לא עקבי.

```
check ((status='settled' and settled_at is not null)
or (status='open' and settled_at is null))
```

Views 3.4

תפקיד	View
אימון + שם המחבר + מספר סטים + נפח + שרירים – בשאילתה אחת	workout_feed
אגרגציה לפי חבר / חבורה / שבוע	weekly_user_stats
מי בחוץ עכשיו – מסווג על closed_at is null and expires_at > now()	active_arrivals

שלושתן security_invoker = on. **זה קריטי:** view רגיל רץ בהרשאות ה-owner ולכן **עוקף RLS** על כל טבלה שמתחתיו. view על workouts שנכתב בבירור המחדל היה מגיש לכל אחד את האימונים של כל החבורות – עקיפה מלאה שנוצרת מעצם הוספת view.

4. הרשאות

שלוש שכבות, בלתי תלויות:

1. GRANT – לאילו טבלאות ועמודות בכלל יש גישה
2. RLS POLICY – אילו שורות מתוך אלה
3. FUNCTION – מי בכלל רשאי לבצע פעולה מורכבת

תפקיד	מה מותר
anon	כלום. אפס הרשאות טבלה
authenticated	SELECT לפי policy; UPDATE על עמודות נבחרות בלבד
service_role	הכול. קיים רק בשרת

4.1 שתי חורים שנסגרו במהלך הפיתוח

א. `UPDATE workouts` היה מאפשר `SET score = 100`. ה-policy "עדכן את האימון שלך" נכונה. ה-GRANT היה רחב מדי. התיקון אינו policy טובה יותר אלא הרשאה ברמת עמודה:

```
grant update (title, notes, performed_at, duration_min) on workouts to authenticated;
```

לעמודה `score` פשוט אין הרשאת UPDATE. מותר לתקן מה שרשמת; אסור לתת לעצמך ציון. אותו דבר על `profiles.creatine_balance`.

ב. ה-views קיבלו הרשאה ל-0002. `anon` הריץ `REVOKE ALL ON ALL TABLES`, אבל `ALL TABLES` חל רק על מה שקיים באותו רגע – וה-views נוצרו ב-0006, ארבע מיגרציות אחר כך. ברירות המחדל של Supabase העניקו אותן מחדש. הפרצה לא הפכה לדליפה כי `security_invoker` עדיין בדק את הרשאות הקורא על הטבלאות שמתחת, ו-`anon` אין לו כאלה. שתי בקורות בלתי תלויות, אחת שגויה, השנייה החזיקה. תוקן ב-0008 פעמיים: `revoke`, ושינוי ברירת המחדל.

5. זרימת מידע

5.1 קריאה

בקשה → `middleware` (`session` רענון, `cookies` כתיבת) → RSC (Server Component)
של המשתמש JWT-עם ה `queries.ts` → Supabase client
→ PostgreSQL → Postgres מפעיל RLS
→ HTML מוגמר לדפדפן

אין API פנימי לקריאות. ה-Server Component נישא ישירות. אין `/api/feed` שצריך להגן עליו בנפרד, כי אין endpoint כזה.

5.2 כתיבה

Client Component → Server Action
├─ Zod: ולידציה
├─ במידת הצורך (`lib/domain`) לוגיקה עסקית
└─ SECURITY DEFINER function → של המשתמש client-דרך ה RPC
├─ `user_id := auth.uid()` ← אף פעם לא מפרמטר
├─ בדיקת הרשאה
└─ כל הכתיבות בטרנזקציה אחת
→ `revalidatePath` → RSC מחדש נטען

5.3 שלושה clients, בכוונה

Client	מפתח	RLS	מתי
browser	publishable	חל	auth state בלבד
server	publishable + JWT	חל	ברירת המחדל לכל דבר
admin	service_role	עוקף	webhook, סגירת שבוע, seed

ל- `admin.ts` יש שמירה שזורקת אם הוא נטען בדפדפן – ייבוא שלו ל-Client Component היה שולח את מפתח ה- `service role` ללקוח, וזו הטעות הגרועה ביותר שאפשר לעשות ב-`stack` הזה.

6. ה-API

הבריק שואל אילו routes או server actions נדרשים. התשובה: בעיקר **Server Actions**, ושני **routes** בלבד.

6.1 Server Actions

Action	קלט	מה עושה
<code>signUpAction</code>	email, password, name	הרשמה
<code>signInAction</code>	email, password	התחברות
<code>signOutAction</code>	—	התנתקות
<code>createHavuraAction</code>	name	<code>create_havura()</code>
<code>joinHavuraAction</code>	code	<code>join_havura()</code>
<code>switchHavuraAction</code>	havurald	החלפת חבורה פעילה
<code>leaveHavuraAction</code>	havurald	עזיבה
<code>announceArrivalAction</code>	havurald, status, note	<code>announce_arrival()</code>
<code>closeArrivalAction</code>	havurald	<code>close_arrival()</code>
<code>logWorkoutAction</code>	[title, when, minutes, sets]	<code>log_workout()</code>
<code>deleteWorkoutAction</code>	workoutId	מחיקה (RLS מגבילה לשלי)
<code>searchExercisesAction</code>	term	חיפוש בקטלוג
<code>purchaseAction</code>	itemId	<code>purchase_shop_item()</code>
<code>equipAction</code>	itemId	<code>equip_item()</code>
<code>startCheckoutAction</code>	packSlug	פתיחת Checkout Session

Route Handlers 6.2

למה דווקא route	Route
חייב להיות ניתן לקריאה מבחוץ (curl, monitoring, E2E)	GET /api/health
Stripe קוראת אליו. Server Action לא ניתן לקריאה מבחוץ	POST /api/stripe/webhook

6.3 פונקציות מסד

פונקציה	הרשאה
create_havura , join_havura , log_workout , purchase_shop_item , equip_item , announce_arrival , close_arrival	authenticated
apply_creatine , compute_workout_score , generate_invite_code	פנימיות – הורשו רק ל- service role
is_havura_member , is_havura_owner , shares_havura_with	עוזרות ל-policies
handle_new_user	טריגר

7. פעולות CRUD

ישות	Create	Read	Update	Delete
חבורה	create_havura()	RLS: רק שלי	בעלים בלבד	בעלים בלבד
חברות	join_havura()	חברים	תפקיד – בעלים	עזיבה או הרחקה
אימון	log_workout()	כל החבורה	עמודות נבחרות, שלי בלבד	שלי בלבד
סטים	דרך log_workout()	כל החבורה	–	cascade
קריאטין	apply_creatine()	שלי בלבד	בלתי אפשרי – append-only	בלתי אפשרי
מלאי	purchase_shop_item()	שלי	equip_item()	–
הזמנה	startCheckoutAction	שלי	webhook בלבד	–
הגעה	announce_arrival()	כל החבורה, בזמן החלון	close_arrival() / אוטומטית ברישום	–

שלוש עמודות ריקות במכונן: ה-ledger לא ניתן לעדכון ולא למחיקה, בשום מסלול. זו התכונה שמגדירה אותו.

8. הלוגיקה העסקית

כולה ב- src/lib/domain/ , טהורה, ללא DB וללא שעון.

8.1 ציון האימון – `scoring.ts`

ק"ג לתרגילי משקל גוף $proxy = 30$ $(proxy + \text{משקל}) \times \text{עבודה לסט} = \text{חזרות}$
נפח = סכום העבודה

Load 0–50 : $25 \times \min(2, \text{נפח} \div \text{חציון}(\text{נפח}))$
Density 0–30 : $15 \times \min(2, \text{חציון}(\text{צפיפות}), \text{צפיפות})$
Coverage 0–20 : עד 20, לכל שריר ראשי שונה, 4

- החציונים מחושבים על 8 האימונים הקודמים של אותו מתאמן. זה כל העניין: הציון יחסי לאדם, לא למשקל מוחלט.

- ללא היסטוריה היחסים הם 1 – ציון ניטרלי, לא מחמיא ולא מעניש.

- חציון בשיטת `percentile_cont` – עם מספר זוגי של דגימות מבצעים אינטרפולציה. `[10, 20, 30, 40]` הוא 25, לא 20. אי-התאמה כאן הייתה שוברת את הזהות מול ה-SQL.

- `proxy` של 30 ק"ג הוא קירוב מוצהר: אנחנו לא אוספים משקל גוף. מופיע כמגבלה ידועה ב-`04-scale.md`.

הפונקציה קיימת פעמיים – ב-`plpgsql` (סמכותית) וב-`TypeScript` (תצוגה חיה). שכפול מכוון: המסד חייב לחשב כדי שלקוח לא יוכל לתת לעצמו 100, וה-`TypeScript` חייב להתקיים כדי שהמתאמן יראה מספר לפני שהוא שומר. `tests/domain.integration.test.ts` מצמיד את השתיים על שבעה מקרים מול המסד האמיתי.

8.2 סגירת תחרות – `competition.ts`

- דירוג תחרותי סטנדרטי: שני ראשונים שווים → שניהם 1, הבא הוא 3.

- תיקו מאחד נתחים. שניים שחולקים מקום 1 תופסים את מקומות 1 ו-2, מאחדים $0.7+0.2$ ומחלקים – 45% כל אחד. אחרת הראשון לפי סדר אלפביתי היה זוכה ב-70%.

- `largest remainder` – הזכיות הן מספרים שלמים שסכומם בדיוק הקופה. עיגול נאיבי היה ממציא או מאבד קריאטין.

- פחות משלושה משתתפים → הנתח הלא-נדרש מחולק מחדש; החבורה תמיד משלמת את מה שהתחייבה.

- מי שלא התאמן מדורג אבל לא מקבל.

8.3 גבולות שבוע – `time.ts`

השבוע רץ ראשון-שבת בשעות ישראל. אימון ב-01:00 ביום ראשון בישראל הוא 22:00 בשבת ב-UTC; חלוקה לפי תאריך UTC הייתה משייכת אותו לשבוע שכבר נסגר. זה לא היפותטי – בדיוק הבאג הזה קרה במערכת הפרודקשן של HavuGym.

8.4 אות העדר – `arrival.ts`

הפונקציה `herdSignal()` היא כל המנגנון מסעיף 2.2 במסמך המוצר, בקוד:

- מסננת הגעות שפגו לפי הזמן שהועבר אליה (טהורה, ולכן ניתנת לבדיקה בכל נקודת זמן).
- מוציאה את הצופה מהספירה – העדר הוא אנשים אחרים.
- מקדימה מי שכבר במכון על פני מי שבדרך: עובדה חזקה מכוונה.
- מסלימה את הניסוח לפי המספר, ומפסיקה לנקוב בשמות מעל שניים.

לא מודל. שני כללים קריאים: השריר שהוזנח הכי הרבה זמן, וקצב מול האתגר לפי הימים שחלפו – כך שיום שלישי ויום שישי נותנים עצה שונה.

9. גבול הקליטה – Heavy כמתאם

במערכת הפרודקשן, נתוני האימונים מגיעים מ-Heavy, צד שלישי, דרך endpoint לא מתועד. זו תלות קיומית בצד שלישי שאיננו שולטים בו.

התכנון כאן הופך את היחס:

```
workouts.source = 'manual' | 'heavy'
↓
feed → צורה מנורמלת אחת → ציון
```

manual הוא המתאם הראשי ועובד היום. heavy יעבוד דרך ה-API הרשמי עם מפתח של המשתמש עצמו. האינדקס הייחודי החלקי על (user_id, source, external_id) כבר קיים, כך שייבוא חוזר לא יוכל לייצר כפילות – גם לפני שהמתאם נכתב.

לא מומש בגרסה זו. מוצהר ב-04-scale.md.

10. ניהול State

סוג	היכן	למה
נתוני שרת	RSC, ללא cache	תמיד טרי; אין state client לסנכרן
טופס בשליחה	useActionState / useFormStatus	pending ושגיאות בלי state ידני
טיוטת אימון	useState ב-workout-logger.tsx	ה-state המורכב היחיד באפליקציה
חבורה פעילה	cookie	שורד ניווט; מאומת מול חברות אמיתית בכל קריאה
בחירת lightbox/tab	useState	מקומי

אין Redux, אין Zustand, אין React Query. ברירת המחדל היא שהשרת מחזיק את ה-state; רק הטיטה בטופס הרישום חיה בלקוח, כי היא באמת לא קיימת בשרת עד השמירה.

11. טיפול בשגיאות

Server Actions לעולם לא זורקות ללקוח. הן מחזירות union:

```
type ActionResult<T> =
  | { ok: true; data: T }
  | { ok: false; error: string; fieldErrors?: Record<string, string> }
```

למה: שגיאה שנזרקה מ-Server Action מגיעה לדפדפן כ-"an error occurred" עם ההודעה מוסרת ב-production – חסר תועלת למשתמש וחסר תועלת לנו. Union מכריח את TypeScript לוודא שכל כישלון מטופל.

פונקציות המסד זורקות **מחרוזות יציבות ומכוונות** (, `INSUFFICIENT_CREDIT`, `NOT_A_MEMBER`, `INVALID_INVITE_CODE`), ו-`friendlyDbError()` ממפה אותן לעברית של בני אדם – בלי להתאים מחרוזות של Postgres עצמו.

שגיאה שהיא מכוונת עמומה: התחברות כושלת מחזירה תמיד "אימייל או סיסמה שגויים". הבחנה בין "אין חשבון" ל-"סיסמה שגויה" הופכת את טופס ההתחברות ל-oracle למניית חשבונות.

12. ולידציה – שלוש שכבות

1. HTML	<code>type, min, max, required</code>	→ משוב מיידי, לא הגנה
2. Zod	<code>Server Action</code> בשרת, בכל	→ הודעה קריאה לאדם
3. CHECK / RPC	→ הקו האחרון	במסד

השכבה השלישית היא היחידה שאי אפשר לעקוף. אם Zod היה נעלם לגמרי, המסד עדיין היה דוחה חזרות שליליות, משקל מעל 500, אימון בלי סטים ויתרה שיוורדת מתחת לאפס. Zod קיים כדי להפוך קלט שגוי להודעה – לא כדי להיות מה שעומד בין המערכת לשורה מושחתת.

13. חוויית המשתמש

- **Mobile-first.** ניווט תחתון בנייד, עליון בדסקטופ. המוצר נפתח בחדר כושר.
- **ערכת נושא כהה אחת, במכוון.** תאורה גרועה, מסך טלפון, ומצגת על מקרן. תמיכה בשתיים מכפילה שטח בדיקה ולא קונה כלום.
- **סט חדש יורש את הקודם.** סטים חוזרים על עצמם הרבה יותר משהם משתנים – זה ההבדל בין ארבע הקלדות לשש-עשרה.
- **הציון החזוי מוצג לפני השמירה,** מפורק ל-`Load / Density / Coverage`. משתמש שרואה מספר בלי הסבר לא מאמין לו.
- **אין אמוג'י.** כל האייקונים הם SVG – אמוג'י נראה שונה בכל פלטפורמה ומוקרא בקול כשם היוניקוד שלו.
- **aria-label על כל שדה סט,** ולכן ה-E2E יכול לתפוס אותם לפי תפקיד – נגישות ובדיקות הן אותה עבודה.

14. ספריות ושירותים חיצוניים

חבילה	למה דווקא היא
next 16	נדרש בבריף. App Router נותן RSC + Server Actions, ולכן אין API פנימי להגן עליו
typescript	נדרש. strict
@supabase/supabase-js	ה-client הרשמי
@supabase/ssr	הטיפול ב-cookies של session ב-RSC. ידני זה שגיאה מובטחת
zod	ולידציה שמייצרת גם טיפוסים – סכמה אחת, לא שתיים
stripe	תשלומים. הדף מתארח אצלם, ולכן אין פרטי כרטיס בשום מקום אצלנו
server-only	הופך ייבוא של קוד שרת ל-Client Component לשגיאת build
tailwindcss 4	עיצוב ליד ה-markup; אין קובץ CSS שגדל לנצח
vitest	מהיר, ESM נטיבי, אותו config כמו Vite
@playwright/test	E2E מול הפריסה האמיתית
pg	בדיקות ו-db:verify בלבד. האפליקציה לא נוגעת בו

מה שלא נכנס, במכוון: ORM (Prisma/Drizzle) – היה מסתיר את ה-RLS ואת ה-RPC, שהם עיקר התכנון; ספריית state – אין state גלובלי; ספריית 12 – UI קומפוננטות לא מצדיקות תלות; ספריית תאריכים – Intl מספיק.